

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 6: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions live in `starter_code.py` at the week root: `generate_array`, `generate_sorted_array`, `generate_reverse_array`, `is_sorted`, `count_inversions_brute`, and the `Counter` class (a tiny mutable operation/comparison counter).

The 12 problems are grouped into three themes:

- **Sorting & Inversions** (Problems 1–5): merging, mergesort with a comparison counter, counting inversions in $O(n \log n)$, a Master Theorem classifier, and empirically confirming mergesort’s recurrence.
- **Recurrences** (Problem 11): comparing the multiplication counts of Karatsuba ($3T(n/2)$) against grade-school divide-and-conquer ($4T(n/2)$).
- **Searching & Selection** (Problems 6–10, 12): binary search, maximum subarray, fast exponentiation, quickselect, majority element, and 1-D peak finding.

1 Sorting and Inversions

Problem 1 – Merge Two Sorted Lists

Implement `merge(left, right)`: given two lists `left` and `right`, each sorted in non-decreasing order, return a single new sorted list containing exactly the elements of both (the multiset union). Use the two-pointer technique: repeatedly compare the front elements of the two lists and append the smaller, then append any remaining tail. Do **not** call `sorted()` on the concatenation.

Example. `merge([1, 3, 5], [2, 4, 6]) == [1, 2, 3, 4, 5, 6]`.

Problem 2 – Mergesort with a Comparison Counter

Implement `mergesort(arr, counter=None)`: return a *new* sorted list (do not mutate `arr`), using the recursive divide-and-conquer mergesort. If a `Counter` (from `starter_code.py`) is passed, call `counter.tick()` once per element comparison performed during merging, so the total reflects the algorithm's comparison count. Verify that `mergesort` agrees with `sorted` and that the comparison count is $O(n \log n)$ (it never exceeds $n \lceil \log_2 n \rceil$).

Problem 3 – Count Inversions in $O(n \log n)$

Implement `count_inversions(arr)`: return the number of inversions (pairs of indices $i < j$ with `arr[i] > arr[j]`) in $O(n \log n)$ time, by modifying mergesort. During each merge, when an element of the right half is emitted before the left half is exhausted, add the number of remaining left-half elements to the inversion count. Verify against `count_inversions_brute` from `starter_code.py`.

Example. `count_inversions([2, 4, 1, 3, 5]) == 3` (the pairs (2,1), (4,1), (4,3)).

Problem 4 – Master Theorem Classifier

Implement `master_theorem(a, b, d)`: for the recurrence $T(n) = aT(n/b) + \Theta(n^d)$ (with $a \geq 1$, $b > 1$, $d \geq 0$), return a string naming the Θ -class. Compare d with $\log_b a$:

- if $d < \log_b a$ (Case 1): return `"Theta(n^{log_b(a)})"` with the numeric exponent, e.g. `"Theta(n^1.585)"`;
- if $d = \log_b a$ (Case 2): return `"Theta(n^d log n)"` with the numeric d , e.g. `"Theta(n^1 log n)"`;
- if $d > \log_b a$ (Case 3): return `"Theta(n^d)"`, e.g. `"Theta(n^2)"`.

Use a small tolerance (10^{-9}) when comparing d to $\log_b a$, and round printed exponents to 3 decimals. The exact string format is given in the starter file's tests.

Problem 5 – Empirically Confirm Mergesort's Recurrence

Implement `count_merge_calls(arr)`: run mergesort instrumented to count the number of *recursive MERGESORT calls* and the number of *MERGE operations* (i.e. non-base-case calls), returning `(num_mergesort_calls, num_merge_ops, total_comparisons)`. Confirm empirically that on an array of length n , the number of MERGE operations equals $n - 1$ (each merge combines two pieces, and n leaves require $n - 1$ internal merges), matching the recurrence structure $T(n) = 2T(n/2) + \Theta(n)$.

Hint. A binary recursion tree with n leaves has exactly $n - 1$ internal nodes.

2 Searching and Selection

Problem 6 – Binary Search (Recursive)

Implement `binary_search(arr, target)`: given a *sorted* list `arr`, return an index i with `arr[i] == target`, or `-1` if `target` is absent. Use the recursive divide-and-conquer formulation (compare with the middle element; recurse into the left or right half). The recurrence is $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$.

Example. `binary_search([1, 3, 5, 7, 9], 7) == 3`; `binary_search([1, 3, 5, 7, 9], 4) == -1`.

Problem 7 – Maximum Subarray Sum (Divide and Conquer)

Implement `max_subarray(arr)`: return the maximum sum of any *contiguous* non-empty subarray of `arr` (which may contain negative numbers), using divide and conquer. Split at the midpoint; the answer is the max of (i) the best subarray in the left half, (ii) the best in the right half, and (iii) the best subarray *crossing* the midpoint (best suffix of the left joined to best prefix of the right). Verify against an $O(n^2)$ brute force.

Example. `max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]) == 6` (the subarray `[4, -1, 2, 1]`).

Problem 8 – Fast Exponentiation (Exponentiation by Squaring)

Implement `fast_pow(x, n, counter=None)`: compute x^n for an integer base x and non-negative integer exponent n , using exponentiation by squaring. If a `Counter` is passed, call `counter.tick()` once per integer multiplication, so the total is $\Theta(\log n)$ (at most about $2\log_2 n$). Verify the result equals `x ** n` and that the multiplication count is $O(\log n)$.

Example. `fast_pow(2, 10) == 1024`; computing it uses far fewer than 9 multiplications.

Problem 9 – Quickselect: k -th Smallest Element

Implement `quickselect(arr, k)`: return the k -th smallest element of `arr` (0-indexed, so `k == 0` is the minimum), using the partition-and-recurse divide-and-conquer method. A deterministic pivot (e.g. the last element, or the middle element) is acceptable. Do not mutate the caller's list. Verify against `sorted(arr)[k]` on random inputs.

Example. `quickselect([7, 2, 1, 6, 8, 5, 3, 4], 0) == 1` (the minimum); `quickselect([7, 2, 1, 6, 8, 5, 3, 4], 3) == 4`.

Problem 10 – Majority Element (Divide and Conquer)

Implement `majority_element(arr)`: return the value occurring *strictly more than* $\lfloor n/2 \rfloor$ times, or `None` if no such value exists, using divide and conquer. Recursively find each half's majority candidate; the only candidates for the whole are these two, so count each candidate's occurrences across the full range and return whichever (if any) exceeds $n/2$. Verify against direct counting with `collections.Counter`.

Example. `majority_element([3, 3, 4, 2, 3, 3, 3]) == 3`; `majority_element([1, 2, 3, 4])` is `None`.

Problem 12 – Peak Finding in 1-D ($O(\log n)$)

Implement `find_peak(arr)`: return the *index* of any *peak* element – an index i such that `arr[i]` is $\geq$ both its neighbours (out-of-bounds neighbours count as $-\infty$) – in $O(\log n)$ via divide and conquer. Look at the middle element; if its right neighbour is larger, a peak must lie to the right (recurse right); else if its left neighbour is larger, recurse left; otherwise the middle is a peak. Verify the returned index is genuinely a peak.

Example. For `[1, 3, 20, 4, 1, 0]`, index 2 (value 20) is a peak. For a strictly increasing array, the last index is the peak.

3 Recurrences

Problem 11 – Karatsuba vs. Grade-School Multiplication Counts

We do not implement full big-integer multiplication here (that is next week); instead we *count multiplications* to compare two recurrences. Implement `karatsuba_mults(n)` returning the number of single-digit (base-case) multiplications performed by the recurrence $T(n) = 3T(n/2) + O(n)$ with $T(1) = 1$, and `gradeschool_mults(n)` for $T(n) = 4T(n/2) + O(n)$ with $T(1) = 1$. Assume n is a power of 2.

For $n = 2^k$, closed forms are `karatsuba_mults(n) =  $3^k = n^{\log_2 3}$` and `gradeschool_mults(n) =  $4^k = n^2$` . Then implement `compare_growth(max_power)` returning a list of tuples `(n, gradeschool, karatsuba, ratio)` for $n = 2^1, 2^2, \dots, 2^{\text{max_power}}$, where `ratio = gradeschool / karatsuba`; confirm the ratio *grows* with n (Karatsuba's count grows strictly slower).

Example. `karatsuba_mults(8) == 27 ( $3^3$ )`; `gradeschool_mults(8) == 64 ( $4^3$ )`.

4 LeetCode Practice

After completing the problems above, practise this week's divide-and-conquer techniques (merge-sort, inversions, recurrences, binary search, quickselect, fast exponentiation) on the following [LeetCode](#) problems. Difficulty is LeetCode's own label.

- [Sort an Array](#) – *Medium* – merge sort / quicksort.
- [Merge Sorted Array](#) – *Easy* – merge.
- [Merge k Sorted Lists](#) – *Hard* – divide & conquer merge.
- [Count of Smaller Numbers After Self](#) – *Hard* – merge sort (inversions).
- [Reverse Pairs](#) – *Hard* – merge sort (inversions).
- [Maximum Subarray](#) – *Medium* – divide & conquer.
- [Pow\(x, n\)](#) – *Medium* – fast exponentiation.
- [Kth Largest Element in an Array](#) – *Medium* – quickselect.
- [Different Ways to Add Parentheses](#) – *Medium* – divide & conquer.